一、初始卷积神经网络

- 1、一张RGB三通道彩色图片处理成训练数据的过程：读入图片(一张图片在计算机内存中就是一个三维数组)->维度转换(将numpy的[高，宽，通道数]转换为PyTorch的CNN要求的[通道数，高，宽]，同时对像素值进行归一化，将其从0~255的整数除以255，变为0.0~1.0的浮点数)->标准化->加上Batch维度(将[C,H,W]变为[N,C,H,W])
- 2、张量：PyTorch的核心数据结构，就是PyTorch中的多维数组容器，能在GPU上跑，能自动求导。
- 3、一个卷积神经网络可以分为
    - 1、输入层：接受原始数据，就本题处理之后的图像张量。
    - 2、卷积层-特征处理器：使用卷积核在图像上滑动，提取局部特征(浅层的卷积提取低级特征：花瓣的轮廓，叶子的边缘；深层的卷积层提取高级特征：花瓣的数量，花蕊的形状)。
    - 3、激活层：常紧跟卷积层，如ReLU函数，用于引入非线性，提高模型表达能力。
    - 4、池化层：对特征图进行下采样，减少数据量，扩大感受野，同时防止过拟合。就相当于把一张高清大图缩小成一张模糊小图。
    - 5、全连接层-决策分类器：将经过卷积和池化提取出的特征图(三维)展平成一维向量，进行最终的分类决策。就是把前面收集到的所有关于"花瓣形状"，"颜色"的线索汇总，判断这到底是什么花。
    - 6、输出层：分类任务中输出类别概率或得分。
- 4、卷积就像是拿着一个小手电在图片上一次扫过。每次只照亮图片的一小块区域，提取这一小块区域的特征。卷积是一种数学运算(加权求和)，它通过一个固定大小的窗口(卷积核)在输入的数据上滑动，每次滑动时，将窗口内的数据与卷积核的对应元素相乘并求和，最终得出一个输出值。
- 5、数学严格意义上的卷积和神经网络的卷积操作是有区别的，但通常被忽略。数学严格意义上的卷积是先翻转卷积核再进行滑动运算。而神经网络中的卷积是不翻转卷积核，他执行的是互相关，因为神经网络中的卷积核参数是通过反向传播学习出来的，无论是否翻转，网络都可以通过调整参数自动学会正确的特征。
- 6、卷积核就是一个二维或三维的权重矩阵，通常很小。他是特征提取器，不同的卷积核负责提取不同的特征。
- 7、步长是卷积核在输入的图片上每次滑动跨越的像素数。
- 8、池化：
    - 最大池化(Max Pooling)，选取窗口内最大值作为输出。提取最强烈的特征，能保留图像的显著特征。
    - 平均池化(Average Pooling)，计算窗口内所有的平均值作为输出。提取整体背景特征，平滑效果好，但容易模糊掉尖锐特征。
    - 全平均池化(Global Average Pooling，GAP)，对整个特征图求平均，直接输出一个值。常用于替代最后的全连接层，极大减少参数量。
    - 自适应池化(Adaptive Pooling)，制定输出尺寸，PyTorch自动计算窗口大小和步长。用于处理不同尺寸的输入。
    - 作用：①降维减参，减少特征图的长度，大幅降低后续计算量和参数量；②扩大感受野，让后面的卷积层能够看到原图上更大的范围；③防止过拟合，坚守特征数量，增强模型的泛化能力；④平移不变性，输入图像发生微小位移时，池化后的输出几乎不变。
- 9、感受野：就是输出特征图上的一个像素点，对应原始输入图像上的区域大小，就是在图片上看到的视野大小。
- 10、激活层的作用是：引入非线性，让神经网络具备拟合复杂函数的能力。
- 11、最后的卷积神经网络会输出各类别的原始得分(Logits)

二、数据预处理

- 1、处理异常/损坏图片：，先创建副本，然后遍历所有的图片，发现损坏的图片，然后将损坏的图片移入一个新的文件夹。
- 2、划分数据集：从train中按照二八分划分出验证集val。
- 3、数据增强，对训练集使用随机数据增强。
- 4、调整图片大小：统一`Resize((224,224))`

三、炼丹

1、回答概念问题

- 1、超参数：在模型训练开始之前人为设定的参数，它们不能通过模型训练自动学习出来。
- 2、超参数如何设定：(1)根据经验，(2)网络搜索，设定多个候选值，挨个训练，找到最好的(3)随即搜索，在参数空间中随即采样，(4)贝叶斯优化，基于历史评估结果，只能预测下一个最有可能提升性能的参数组合。
- 3、优化器：决定如何更新网络权重参数的算法，它的核心是让损失函数(Loss)最小化。
- 4、Dataset是负责单个样本的读取和处理，它定义了如何获取一张图片以及对应的标签。
- 5、DataLoader是负责批量处理，它从Dataset中去除多个样本，打包成一个Batch，并支持打乱(Shunffle)、多进程并行加载(num_workers)和GPU内存锁定(pin_memory)。

四、选做

- 1、并不是模型的层数越多、结构越深，性能就一定越好。原因：1、梯度消失/爆炸，层数太多，反向传播时梯度会变成0或者无穷大，导致前面的层根本学不到东西。2、网络退化，即使解决了梯度问题，盲目加深网络也会导致训练误差不降反升，深层网络很难优化。所以这就是我们为什么需要引入现代网络结构(如ResNet)来打破这个限制。
- 2、模型对比：在src\train_pretrained.py。
  - 实验数据对比：自建CNN模型：训练20个Epoch，单轮耗时约11.5秒，最终测试集准确率为77.44%。
  - ResNet18微调模型：训练仅5个Epoch，单轮耗时约9秒，验证集准确率即达到 85.26%。
- 3、我们不需要每次使用前都重新训练。在`train.py`中，使用了`torch.save(model.state_dict(), 'best_model.pth')`将训练好的权重保存到了本地。在`predict.py`中，通过`model.load_state_dict(torch.load('best_model.pth'))`直接加载权重。